

SIMATS ENGINEERING
(SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering

ASSESSMENT REPORT

Intelligent Warehouse Inventory Platform

Code Review, Repository Evaluation & Architectural Assessment

Field	Details
Course	Software Engineering
Course Code	CSA1001
CO Assessed	ASSESSMENT4– Full-stack applications, containerization, version control and collaboration
Assessment	Code Review & Repository Evaluation
Student Name / Reg. No.	G.KOUSHIK(192524238)
Date	

Prepared according to the supplied assessment brief and rubric.

1. Problem Overview and System Objectives

The Intelligent Warehouse Inventory Platform is a full-stack software system for managing warehouse stock, product information, storage locations, inventory movements, low-stock conditions and operational visibility. It replaces fragmented manual tracking with a centralized application that can support warehouse staff and administrators.

The supplied assessment requires analysis of the problem and solution, repository organization, source-code quality, Git practices, testing, documentation, findings and recommendations. This report addresses those areas while presenting the architecture and component design expected for an engineering assessment.

1.1 Objectives

- Maintain a centralized inventory catalogue containing product identifiers, names, categories and quantities.
- Track stock-in and stock-out operations and maintain an auditable movement history.
- Identify low-stock items using configurable reorder thresholds.
- Provide dashboard visibility into inventory status and warehouse activity.
- Separate frontend, backend and data-access responsibilities for maintainability.
- Validate inputs and handle errors consistently.
- Support collaborative development through Git and prepare the application for containerized deployment.

1.2 Key Functionalities

Module	Purpose	Expected Output
Dashboard	Summarize stock health and activity	KPIs, alerts and recent movements
Product Management	Create, view, update and organize products	Accurate product catalogue
Inventory Operations	Record additions and removals	Updated quantities and movement log
Warehouse / Location	Associate items with storage areas	Location-aware inventory
Alerts	Detect threshold breaches	Low-stock indicators
Reports / History	Review inventory changes	Traceable operational records

1.3 Assessment Alignment

The rubric allocates marks to problem analysis and repository organization, code quality, version control, testing and validation, documentation and maintenance, and review findings/presentation. Therefore, the report connects implementation decisions with evidence and maintainability rather than treating them as isolated topics.

2. Software Architecture and Component Design

A layered client–server architecture is recommended. The presentation layer handles user interaction, the application/API layer validates requests and applies business rules, and the persistence layer manages inventory data. This separation supports modularity, testing and future containerization.

2.1 Proposed Architecture

Layer	Main Components	Responsibility
Presentation	Web UI, dashboard, forms, tables	Display data and collect input
API / Application	Routes, controllers, services	Request handling and business rules
Domain	Product, Inventory, Location, Alert models	Represent warehouse concepts and rules
Persistence	Repositories / database access	Store and retrieve inventory data

2.2 Component Interaction

A typical inventory update begins when a user submits a transaction in the web interface. The frontend performs basic validation and sends the request to the API. The service layer verifies product and quantity rules, the persistence layer updates inventory state and records the movement, and the API returns the result to the interface.

[ARCHITECTURE DIAGRAM]

User → Web UI → REST API → Service Layer → Repository/Data Layer → Inventory Database
↘ Validation / Alerts / Audit History ↗

Insert a polished architecture diagram or screenshot of the implemented architecture in this location.

2.3 Quality Attribute Analysis

Attribute	Design Approach	Assessment Value
Maintainability	Modular services and clear naming	Changes can be isolated
Reliability	Validation, error handling and controlled updates	Reduces inconsistent stock states
Usability	Dashboard, search/filter and clear feedback	Improves warehouse workflow
Scalability	Separated API and persistence layers	Supports future growth
Security	Authentication, authorization and input validation	Protects operational data

3. Repository Organization and Code Quality Review

The repository should be easy for a new contributor to understand. A predictable structure separates application responsibilities, configuration, documentation and tests. Actual screenshots and repository identifiers should be inserted from the student's implementation.

3.1 Recommended Repository Structure

```
warehouse-inventory/
├── frontend/
│   ├── src/
│   ├── public/
│   └── package.json
├── backend/
│   ├── src/
│   │   ├── controllers/
│   │   ├── services/
│   │   ├── models/
│   │   ├── repositories/
│   │   └── routes/
│   └── package.json
├── tests/
├── docs/
├── .gitignore
├── README.md
├── Dockerfile
└── docker-compose.yml
```

3.2 Code Quality Review

Area	Good Practice	Evidence to Capture	Benefit
Readability	Descriptive names and consistent formatting	Representative code screenshot	Faster review
Modularity	Controllers/services/repositories separated	Folder/code screenshots	Lower coupling
Validation	Required and range checks	Validation code + result	Fewer invalid states
Error Handling	Consistent API errors and UI messages	Error screenshot	Better reliability
Documentation	README explains setup and usage	README screenshot	Easier onboarding
Maintainability	Reusable components and functions	Code example	Less duplication

3.3 Review Findings

- The layered structure is appropriate because inventory rules should not be mixed directly into UI components.
- Reusable product and inventory components reduce duplicated interface logic and simplify future changes.
- Validation should occur at client and server boundaries, with server-side validation remaining authoritative.
- Sensitive configuration should be stored through environment variables rather than committed source code.
- The repository should exclude generated dependencies, build output, secrets and local environment files.

Evidence note: insert actual repository-tree and source-code screenshots before final submission because the supplied assessment explicitly expects them.

4. Version Control Evaluation

Git should be the source of truth for collaborative development. A strong repository records focused changes with descriptive commit messages and keeps the main branch stable.

Practice	Recommended Approach	Evidence	Why It Matters
Commit History	Frequent, focused commits	Git log screenshot	Shows incremental development
Commit Messages	Action-oriented descriptions	Sample commit list	Improves traceability
Branches	Feature branches with stable main	Branch screenshot	Reduces integration risk
Pull Requests	Review before merging	PR screenshot, if available	Encourages peer review
Repository Hygiene	README, .gitignore and organized files	Root screenshot	Improves maintainability

5. Testing and Validation

Testing should demonstrate that core warehouse workflows work correctly and that invalid inputs are handled safely. Replace the status placeholders below with actual execution results from the implemented application.

ID	Test Case	Input / Action	Expected Result	Status
TC01	Add product	Valid SKU, name and quantity	Product is created	PASS / UPDATE
TC02	Stock-in	Valid quantity	Quantity increases and movement recorded	PASS / UPDATE
TC03	Stock-out	Quantity within available stock	Quantity decreases correctly	PASS / UPDATE
TC04	Excess stock-out	Quantity above available stock	Operation rejected clearly	PASS / UPDATE
TC05	Low-stock alert	Quantity below threshold	Alert displayed	PASS / UPDATE
TC06	Invalid product	Missing required field	Validation prevents submission	PASS / UPDATE
TC07	Search/filter	Existing SKU/category	Matching records displayed	PASS / UPDATE

Suggested evidence: dashboard screenshot, product-management screenshot, transaction result, validation-error screenshot and test execution output.

6. Documentation and Repository Maintenance

The README should allow a new developer to understand, install, run and test the application without relying on private knowledge. Frontend, backend and deployment assumptions should remain synchronized.

README Section	Required Content	Review Check
Project Overview	Problem, purpose and major features	Present / Improve
Technology Stack	Frontend, backend, database and tools	Present / Improve
Installation	Prerequisites and dependency installation	Present / Improve
Configuration	Environment variables and safe configuration	Present / Improve
Usage Guide	Main workflows and screenshots	Present / Improve
Testing	Commands, cases and expected output	Present / Improve
Deployment	Build/container/run instructions	Present / Improve

7. Review Findings and Recommendations

Area	Recommendation
Architecture	Keep UI, business logic and data access separated and document interactions.
Code Quality	Use consistent formatting, reusable components, meaningful names and centralized validation.
Version Control	Use focused commits, feature branches and review before merging; never commit secrets.
Testing	Automate core inventory tests and include positive and negative cases.
Documentation	Maintain a complete README covering setup, usage, testing and deployment.
Usability	Use clear stock states, search/filter controls, confirmation messages and accessible forms.
Future Enhancement	Consider barcode support, forecasting, supplier integration, role-based access and analytics.

7.1 Overall Assessment

The Intelligent Warehouse Inventory Platform is a suitable full-stack software engineering problem because it contains clear domain entities, transactional workflows, user-facing functionality and opportunities for modular architecture. To reach the highest rubric level, the final submission should support claims with repository evidence, test execution evidence, screenshots and a professional README.

8. Rubric Mapping and Submission Checklist

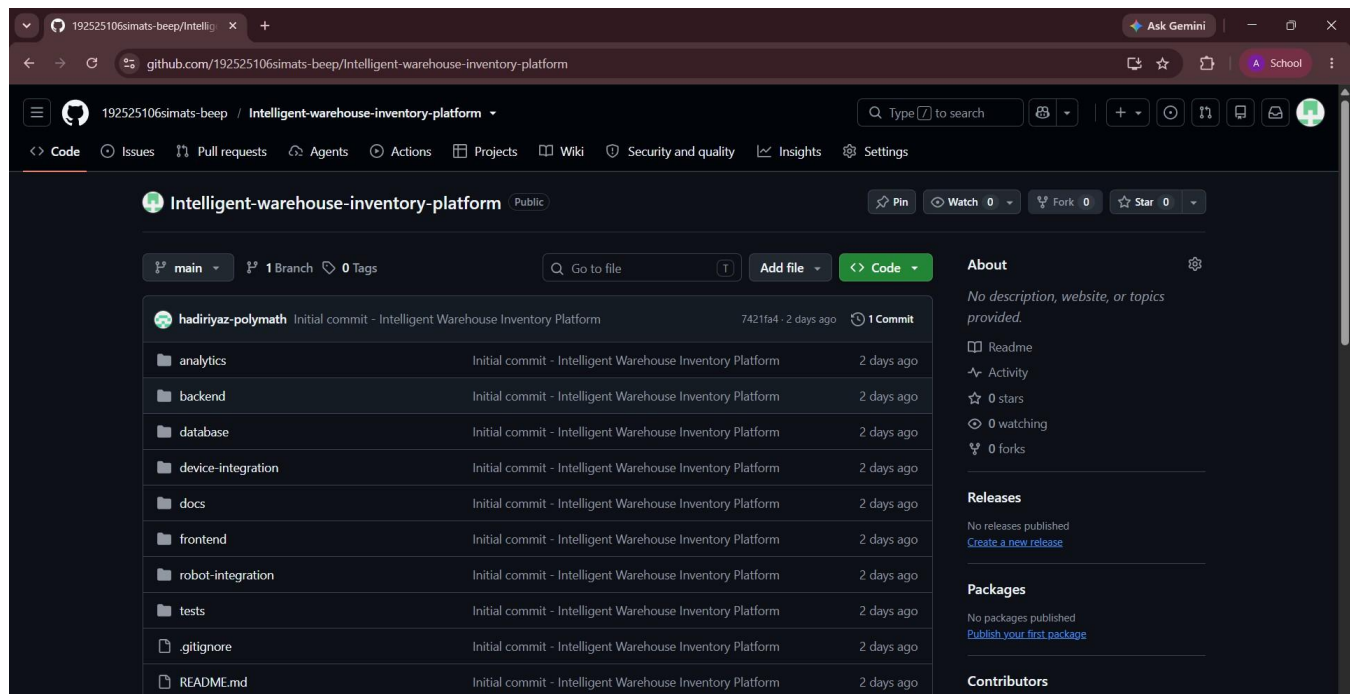
The supplied rubric identifies the following scoring areas. This mapping provides a final pre-submission checklist aligned with the evaluator's expectations.

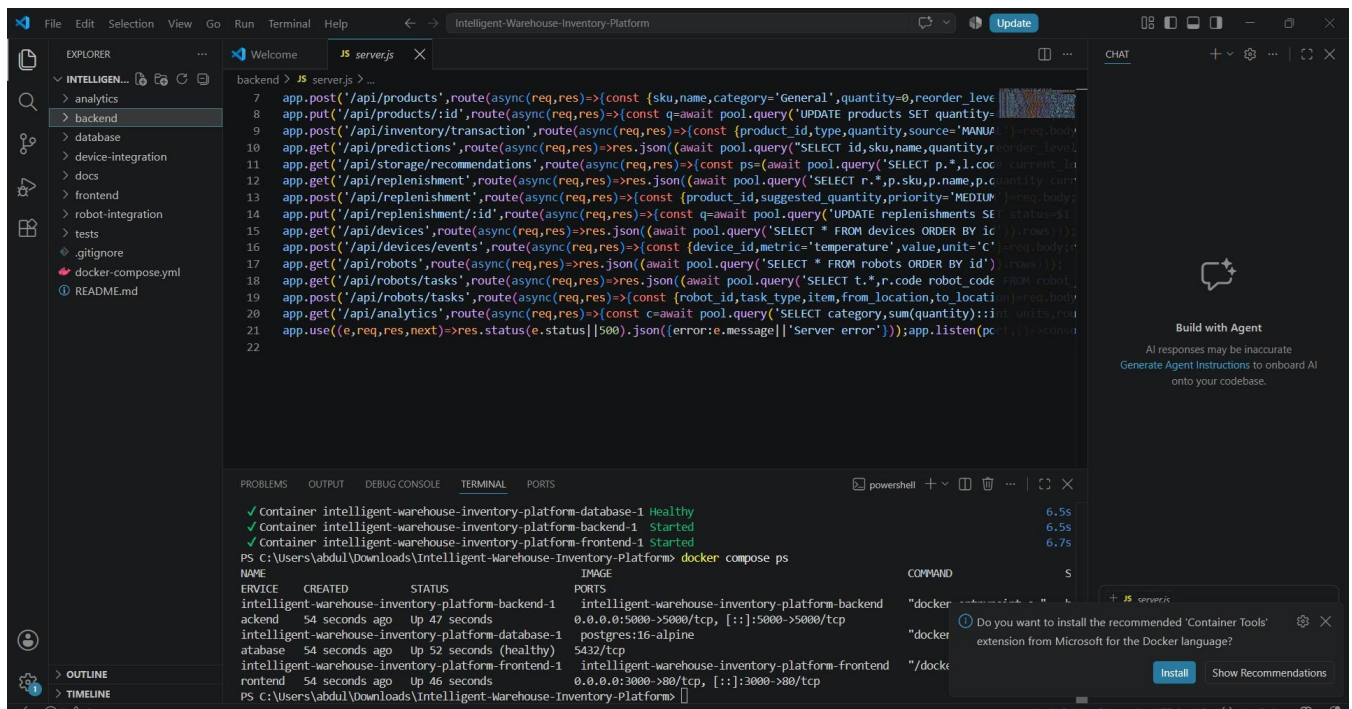
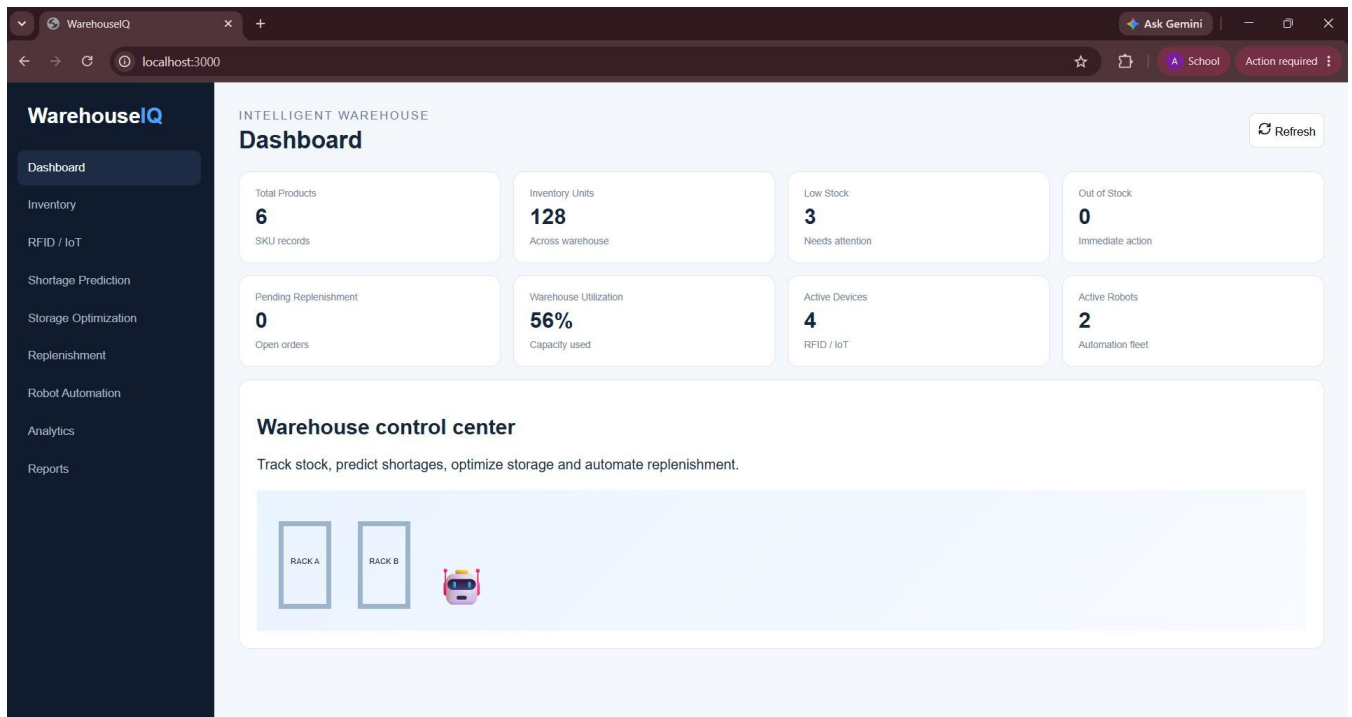
Rubric Area	Weight	Coverage in Report	Evidence to Attach
Problem Analysis & Repository Organization	15%	Problem, objectives, functions and structure	Repository tree screenshot
Code Quality Review	25%	Readability, modularity, standards and maintainability	Code screenshots/snippets
Version Control Evaluation	20%	Commits, branches, messages and maintenance	Git history/branch screenshot
Testing & Validation	15%	Test matrix and validation strategy	Execution screenshots
Documentation & Repository Maintenance	15%	README, installation, usage and deployment	README screenshot
Review Findings, Recommendations & Presentation	10%	Findings, recommendations and professional presentation	Final report/evidence

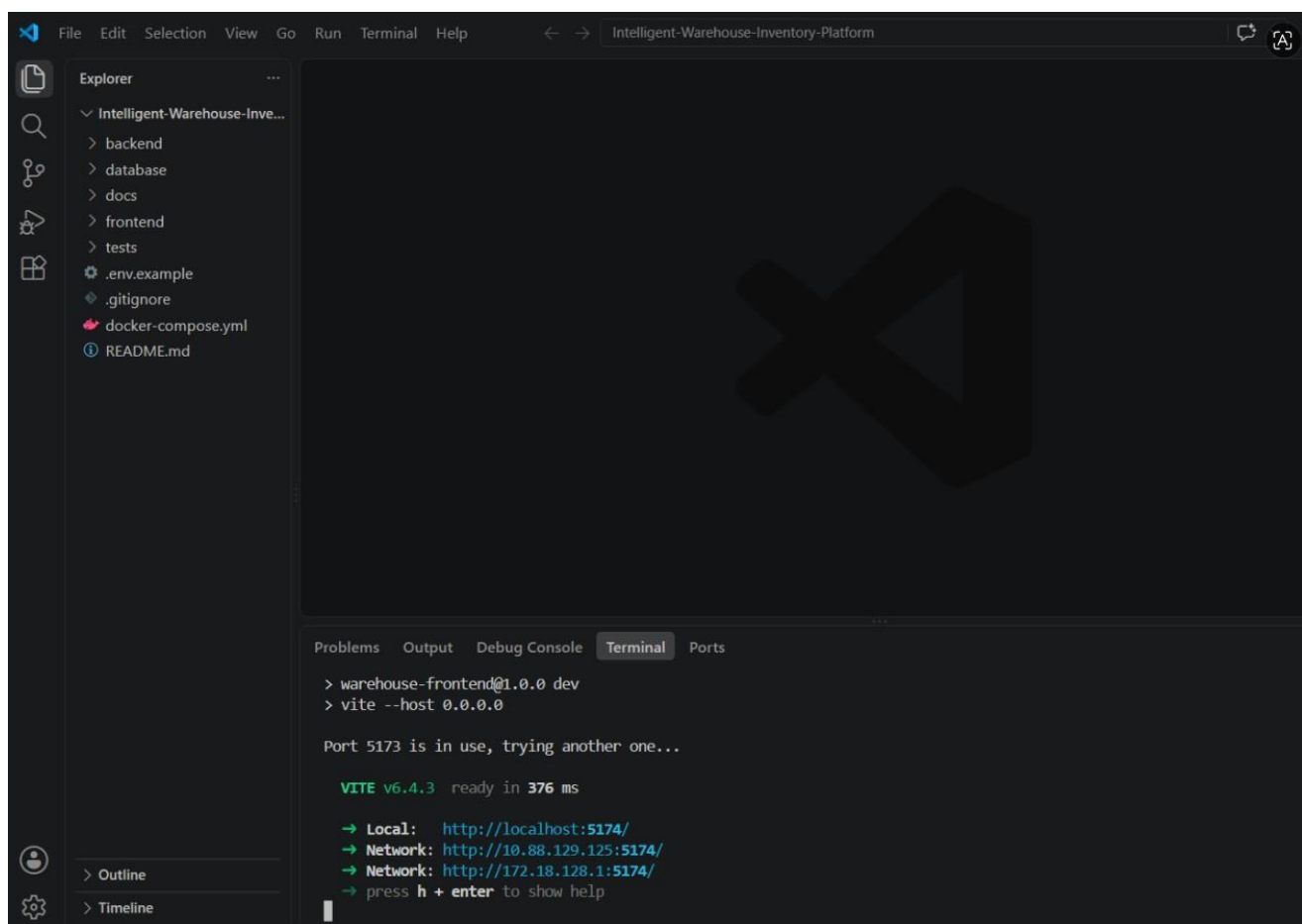
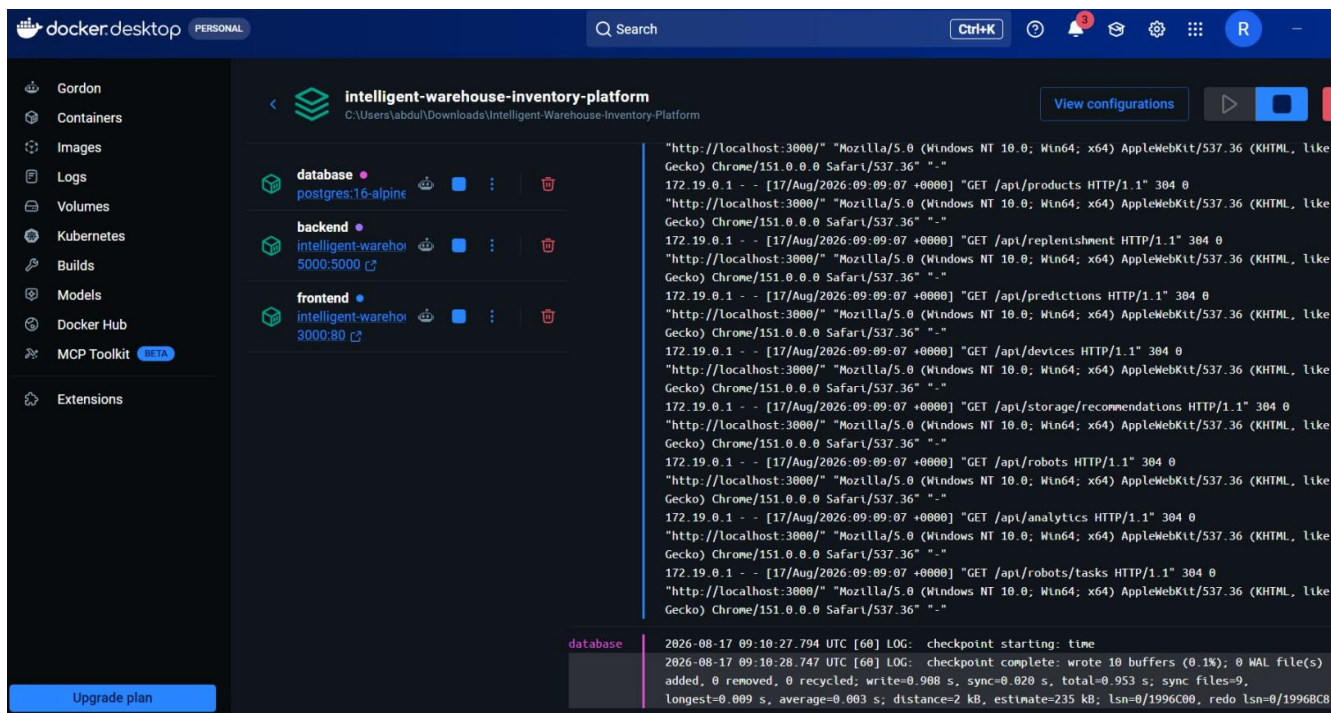
8.1 Final Submission Checklist

- Insert student name, registration number and date.
- Add the actual Git repository URL.
- Insert a screenshot of repository structure.
- Insert a screenshot of commit history and branches.
- Insert representative source-code snippets demonstrating quality and modularity.
- Insert application screenshots showing dashboard, inventory operations and validation.
- Replace all PASS / UPDATE placeholders with actual test results.
- Complete the README and installation/usage instructions.
- Ensure screenshots contain no passwords, API keys or private configuration.
- Proofread headings, captions, references and page numbering.

Code Review & Repository Evaluation







Conclusion

This report provides a rubric-aligned assessment structure for the Intelligent Warehouse Inventory Platform. A high-quality final submission should combine the written analysis with concrete repository and application

evidence. The proposed architecture separates responsibilities, the testing matrix validates critical workflows, and the recommendations focus on maintainability, collaboration, documentation and future scalability.